

École Supérieure de Technologie de Tétouan

Département d'Informatique

Rapport de TP

Exploration de l'espace des états

Recherche Informée

Étudiant(e) :	SEBAGH Badr Eddine
Encadrant / Professeur :	Faouzi Marzouki
Module :	INTRODUCTION IA
Année universitaire :	2025–2026

Table des matières

1	Introduction	2
2	Description des Algorithmes	3
2.1	Concepts et Fonctions Auxiliaires	3
2.2	Algorithmes de Recherche Non-Informée	4
2.3	Algorithmes de Recherche Informée	5
3	Détails d'Implémentation	7
3.1	Architecture Modulaire	7
3.2	Génération de l'Environnement (<code>maze.py</code>)	7
3.3	Protocole de Mesure et Métriques (<code>main.py</code>)	9
3.4	Visualisation des Résultats (<code>visualization.py</code>)	11
4	Résultats et Analyse	13
4.1	Présentation des Résultats Statistiques	13
4.2	Analyse Comparative par Graphiques	14
5	Conclusion	18

1 Introduction

Le problème de la résolution de labyrinthes est un défi classique en **Intelligence Artificielle** et en **théorie des graphes**. Ce projet vise à concevoir et comparer des systèmes capables de naviguer dans des environnements complexes pour trouver un chemin optimal entre un point de départ et une destination.

Objectifs de l'étude

- Réaliser une étude comparative approfondie entre des algorithmes de recherche **non-informés** et **informés**.
- Évaluer l'efficacité de chaque méthode à travers des simulations répétées sur des labyrinthes aléatoires.
- Analyser les performances selon des critères quantitatifs rigoureux : **temps**, **mémoire** et **qualité du chemin**.

Algorithmes étudiés

Cette étude repose sur une comparaison entre quatre stratégies fondamentales :

- **Recherche Non-Informée (Uninformed Search)** :
 - **BFS (Breadth-First Search)** : exploration par couches garantissant le chemin le plus court.
 - **DFS (Depth-First Search)** : exploration en profondeur, rapide mais souvent non optimale.
- **Recherche Informée (Informed Search)** :
 - **A* Search** : utilise une heuristique, notamment la **distance de Manhattan**, pour optimiser la recherche et garantir l'optimalité.
 - **Greedy Best-First Search** : se base uniquement sur l'heuristique pour atteindre la cible le plus rapidement possible.

Métriques de performance

L'analyse se focalise sur quatre indicateurs clés :

- **Vitesse d'exécution** : mesurée via `time.perf_counter()`.
- **Efficacité mémoire** : suivi du pic de consommation avec `tracemalloc`.
- **Optimalité du chemin** : longueur totale du chemin trouvé.

Outils techniques

Le développement a été réalisé en **Python**, en s'appuyant sur un écosystème robuste pour le traitement et la visualisation des données :

- **Pandas** : pour la structuration et la manipulation des résultats expérimentaux.
- **Matplotlib & Seaborn** : pour la génération de graphiques avancés.
- **NumPy** : pour la gestion efficace des structures de la grille.

2 Description des Algorithmes

Cette section détaille la logique interne des algorithmes de recherche implémentés pour la résolution du labyrinthe. Nous distinguons les approches **non-informées**, également appelées **recherche aveugle**, des approches **informées**, qui exploitent une heuristique pour guider l'exploration.

2.1 Concepts et Fonctions Auxiliaires

Deux concepts fondamentaux sont implémentés dans `algorithms.py` :

- **Distance de Manhattan** : pour estimer la distance entre une position actuelle et la sortie, nous utilisons la distance de Manhattan. Elle est calculée comme la somme des différences absolues de leurs coordonnées :

$$d = |x_1 - x_2| + |y_1 - y_2|.$$

```
def manhattan_distance(p1, p2):  
    """Heuristic: Manhattan distance calculation"""  
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
```

- **Reconstruction du Chemin** (`reconstruct_path`) : une fois l'objectif atteint, cette fonction remonte le dictionnaire des parents, noté `came_from`, afin de générer la liste ordonnée des cases constituant le chemin final.

```
def reconstruct_path(came_from, current):  
    """Backtracks from the goal to the start to reconstruct the final path"""  
    path = []  
    while current is not None:  
        path.append(current)  
        current = came_from[current]  
    return path[::-1]
```

2.2 Algorithmes de Recherche Non-Informée

A. Breadth-First Search (BFS)

- **Logique** : BFS explore le labyrinthe niveau par niveau en utilisant une **file** (*Queue* – FIFO).
- **Performance** : cette méthode garantit l'**optimalité**, c'est-à-dire le chemin le plus court, dans une grille où chaque déplacement possède le même coût.

```
def bfs_search(maze):
    """Breadth-First Search (BFS) - Guarantees the shortest path in uniform grids"""
    start = maze.start
    queue = deque([start])
    came_from = {start: None}
    nodes_explored = 0

    while queue:
        current = queue.popleft()
        nodes_explored += 1

        if maze.is_goal(current):
            return reconstruct_path(came_from, current), nodes_explored

        for neighbor in maze.get_neighbors(current):
            if neighbor not in came_from:
                came_from[neighbor] = current
                queue.append(neighbor)
    return None, nodes_explored
```

B. Depth-First Search (DFS)

- **Logique** : DFS explore une branche aussi loin que possible en utilisant une **pile** (*Stack* – LIFO), avant de revenir en arrière par **backtracking**.
- **Performance** : cette méthode est rapide en mémoire, mais elle ne garantit pas de trouver le chemin le plus court, ce qui conduit souvent à des trajets très longs.

```

def dfs_search(maze):
    """Depth-First Search (DFS) - Does not guarantee the shortest path"""
    start = maze.start
    stack = [start]
    came_from = {start: None}
    nodes_explored = 0

    while stack:
        current = stack.pop()
        nodes_explored += 1

        if maze.is_goal(current):
            return reconstruct_path(came_from, current), nodes_explored

        for neighbor in maze.get_neighbors(current):
            if neighbor not in came_from:
                came_from[neighbor] = current
                stack.append(neighbor)
    return None, nodes_explored

```

2.3 Algorithmes de Recherche Informée

A. Algorithme A*

- **Logique** : A* est un algorithme hybride qui utilise une fonction de score :

$$f(n) = g(n) + h(n).$$

Dans cette formule, $g(n)$ représente le **coût réel** du chemin parcouru depuis le départ, tandis que $h(n)$ correspond à l'**estimation heuristique** vers la cible, calculée ici avec la distance de Manhattan.

- **Performance** : A* est à la fois **complet** et **optimal**. Il réduit considérablement l'espace de recherche par rapport à BFS en se dirigeant intelligemment vers le but.

```

def a_star_search(maze):
    """A* Search - Combines path cost and heuristic for optimal efficiency"""
    start = maze.start
    goal = maze.goal
    pq = [(0 + manhattan_distance(start, goal), start)]
    came_from = {start: None}
    g_score = {start: 0}
    nodes_explored = 0

    while pq:
        current_f, current = heapq.heappop(pq)
        nodes_explored += 1

        if maze.is_goal(current):
            return reconstruct_path(came_from, current), nodes_explored

        for neighbor in maze.get_neighbors(current):
            tentative_g_score = g_score[current] + 1
            if neighbor not in g_score or tentative_g_score < g_score[neighbor]:
                g_score[neighbor] = tentative_g_score
                f_score = tentative_g_score + manhattan_distance(neighbor, goal)
                heapq.heappush(pq, (f_score, neighbor))
                came_from[neighbor] = current

    return None, nodes_explored

```

B. Greedy Best-First Search (GBFS)

- **Logique** : contrairement à A*, cet algorithme prend ses décisions uniquement sur la base de l'heuristique $h(n)$.
- **Performance** : GBFS est très rapide en temps de calcul, car il se dirige directement vers la cible. Cependant, il peut être facilement piégé par des obstacles complexes, tels que des murs en U, ce qui peut mener à des solutions **sous-optimales**.

```

def greedy_best_first_search(maze):
    """Greedy Best-First Search (GBFS) - Relies solely on the heuristic"""
    start = maze.start
    goal = maze.goal
    pq = [(manhattan_distance(start, goal), start)]
    came_from = {start: None}
    visited = {start}
    nodes_explored = 0

    while pq:
        current_h, current = heapq.heappop(pq)
        nodes_explored += 1

        if maze.is_goal(current):
            return reconstruct_path(came_from, current), nodes_explored

        for neighbor in maze.get_neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                heapq.heappush(pq, (manhattan_distance(neighbor, goal), neighbor))
                came_from[neighbor] = current

    return None, nodes_explored

```

3 Détails d'Implémentation

Cette section décrit l'architecture modulaire du projet et les outils techniques mis en œuvre pour garantir une évaluation rigoureuse des performances.

3.1 Architecture Modulaire

Le projet est structuré en plusieurs modules Python afin d'assurer une séparation claire des responsabilités :

- `maze.py` : responsable de la génération de l'environnement de test.
- `algorithms.py` : contient les implémentations des quatre algorithmes de recherche.
- `visualization.py` : gère la création des graphiques et la visualisation du chemin final.
- `main.py` : représente le moteur d'exécution qui orchestre les tests et collecte les données.

3.2 Génération de l'Environnement (`maze.py`)

L'environnement de test est une grille 2D, ou **matrice**, générée aléatoirement via la classe `Maze`. Sa structure repose sur les éléments suivants :

- **Initialisation spatiale** : la grille est définie par une `height`, correspondant aux lignes, et une `width`, correspondant aux colonnes. Les points de départ et d'arrivée sont fixés par défaut aux extrémités opposées de la matrice.
- **Densité d'obstacles** : le paramètre `density` définit la probabilité, comprise entre **0** et **1**, qu'une cellule soit un mur, c'est-à-dire bloquée et notée 1, ou un passage libre, noté 0. Une vérification post-génération assure que le départ et l'arrivée ne sont jamais des murs.
- **Gestion des voisins** : la méthode `get_neighbors` implémente la logique de déplacement. Elle analyse les quatre directions adjacentes, **haut**, **bas**, **gauche** et **droite**, et ne retourne que les coordonnées qui sont à la fois dans les limites de la grille et libres de tout obstacle.

```
class Maze:
    def __init__(self, width, height, density=0.3):

        self.width = width
        self.height = height
        self.density = density
        self.grid = []
        self.start = (0, 0)
        self.goal = (height - 1, width - 1)
        self.generate_maze()

    def generate_maze(self):
        """Generates the maze grid based on the specified density"""
        self.grid = []
        for r in range(self.height):
            row = []
            for c in range(self.width):
                # Randomly place a wall (1) or a path (0) based on density
                if random.random() < self.density:
                    row.append(1) # 1 represents a wall
                else:
                    row.append(0) # 0 represents a path
            self.grid.append(row)

        # Ensure that the start and goal points are paths, not walls
        self.grid[self.start[0]][self.start[1]] = 0
        self.grid[self.goal[0]][self.goal[1]] = 0
```

```

def get_neighbors(self, pos):
    """Returns available neighboring nodes (non-walls) for a given position"""
    row, col = pos
    neighbors = []
    # Four movement directions: Right, Left, Down, Up
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    for dr, dc in directions:
        r, c = row + dr, col + dc
        if 0 <= r < self.height and 0 <= c < self.width and self.grid[r][c] == 0:
            neighbors.append((r, c))
    return neighbors

def is_goal(self, pos):
    """Checks if the current position is the goal"""
    return pos == self.goal

```

3.3 Protocole de Mesure et Métriques (main.py)

Pour obtenir des résultats statistiquement fiables, nous avons mis en place un protocole d'exécution de **20 itérations** par algorithme sur des labyrinthes générés aléatoirement. Les outils et métriques suivants ont été utilisés :

- **Mesure temporelle** : utilisation de `time.perf_counter()` pour capturer la durée exacte de l'exécution, exprimée en **secondes**, avec une haute précision.
- **Suivi de la mémoire** : utilisation de la bibliothèque `tracemalloc` pour mesurer le pic de consommation mémoire, ou *peak memory*. Les données sont converties en **mégaoctets (MB)** afin de faciliter l'analyse.
- **Efficacité de recherche (nodes_explored)** : calcul du nombre total de nœuds visités pour évaluer l'*overhead*, c'est-à-dire le **surcroît de travail**, de chaque stratégie avant d'atteindre l'objectif.
- **Qualité de la solution (Path Length)** : mesure de la longueur du chemin final afin de comparer l'**optimalité** des solutions produites par les recherches informées et non-informées.
- **Stockage et traitement** : toutes les données brutes sont centralisées dans un *DataFrame* **Pandas**, permettant de calculer automatiquement la **moyenne** et l'**écart-type** via des fonctions d'agrégation.

```

def run_experiments(algorithms, maze_size=(100, 100), density=0.3, runs=20):
    """ Executes experiments and collects all required performance data """
    results = []
    for name, algo_func in algorithms.items():
        print(f"Running {name}...")
        for i in range(runs):
            m = Maze(maze_size[0], maze_size[1], density)

            tracemalloc.start()
            start_time = time.perf_counter()

            # Returning path and nodes_explored for performance metrics
            path, nodes_explored = algo_func(m)

            end_time = time.perf_counter()
            _, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()

            if path:
                results.append({
                    'Algorithm': name,
                    'Run': i + 1,
                    'Time': end_time - start_time,
                    'Memory': peak / 10**6, # Memory in MB
                    'Path Length': len(path),
                    'Nodes Explored': nodes_explored
                })
    return pd.DataFrame(results)

```

```

if __name__ == "__main__":
    # 1. Define the algorithms to be tested
    algorithms = {
        'BFS': bfs_search,
        'DFS': dfs_search,
        'A*': a_star_search,
        'Greedy': greedy_best_first_search
    }

    # 2. Run the experiment suite
    df_results = run_experiments(algorithms, maze_size=(100, 100), runs=20)

    # 3. Generate statistical summary (Mean & Std)
    summary_stats = df_results.groupby('Algorithm').agg({
        'Time': ['mean', 'std'],
        'Memory': ['mean', 'std'],
        'Path Length': ['mean', 'std']
    }).round(4)

    print("\n" + "="*60)
    print("STATISTICAL RESULTS (MEAN & STD)")
    print("="*60)
    print(summary_stats)

    # 4. Execute all visualization functions
    print("\nGenerating all visualizations...")

    # Bar Plot for metric comparison
    plot_bar_comparison(df_results)

    # Box Plot for result distribution
    plot_box_plots(df_results)

    # Pie Chart for Nodes Explored proportions
    plot_pie_charts(df_results)

    # Line Plot for performance stability across runs
    plot_line_metrics(df_results)

```

3.4 Visualisation des Résultats (visualization.py)

Pour l'interprétation des résultats, nous avons exploité les bibliothèques **Matplotlib** et **Seaborn** à travers cinq types de visualisations complémentaires. Chaque graphique apporte un angle d'analyse spécifique afin de comparer les algorithmes non seulement sur leurs performances moyennes, mais aussi sur leur stabilité, leur dispersion et leur comportement spatial dans le labyrinthe.

- **Bar Plots (Comparaison des Moyennes)** : ces graphiques comparent les moyennes globales du **temps d'exécution** (Time), de l'**utilisation mémoire** (Memory) et de la **longueur du chemin** (Path Length) pour chaque algorithme. Ils permettent d'identifier rapidement

les méthodes les plus efficaces selon chaque métrique principale.

- **Box Plots (Analyse de Distribution)** : les boîtes à moustaches permettent d'étudier la **dispersion** des résultats sur les **20 tentatives**. Elles sont particulièrement utiles pour visualiser la variabilité du temps et de la mémoire, ainsi que pour détecter les **valeurs aberrantes** (*outliers*) causées par certaines configurations de labyrinthes plus difficiles.
- **Pie Charts (Répartition Proportionnelle)** : ces graphiques représentent la part relative de chaque algorithme dans la consommation totale des ressources ou dans l'espace de recherche exploré. Ils offrent une vue synthétique de l'**efficacité comparative** en montrant quels algorithmes dominent proportionnellement l'utilisation du temps, de la mémoire ou du nombre de nœuds explorés.
- **Line Plots (Stabilité de l'Exécution)** : les courbes tracent l'évolution du **temps d'exécution** au fil des répétitions, c'est-à-dire sur les **20 runs**. Cette visualisation est essentielle pour évaluer la **stabilité temporelle** des algorithmes face à la génération aléatoire des grilles et pour repérer d'éventuelles fluctuations importantes d'une exécution à l'autre.
- **Visualisation du Labyrinthe (visualize_maze_solution)** : cette fonction génère une cartographie spatiale de la grille binaire du labyrinthe. Le chemin trouvé est tracé en **rouge**, tandis que le point de départ est marqué en **vert** et l'objectif en **bleu**. Elle permet de vérifier visuellement la validité du chemin produit et d'observer la manière dont la solution traverse les obstacles.

```
def plot_bar_comparison(stats_df):
    """ 1. Bar Plot: Compares averages for Time, Memory, and Path Length """
    metrics = ['Time', 'Memory', 'Path Length']
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    axes = axes.flatten()

    for i, metric in enumerate(metrics):
        sns.barplot(x='Algorithm', y=metric, data=stats_df, ax=axes[i], palette='magma', errorbar=None)
        axes[i].set_title(f'Average {metric}')
        axes[i].set_ylabel(metric)

    plt.tight_layout()
    plt.show()

def plot_box_plots(stats_df):
    """ 2. Box Plot: Visualizes the distribution of Time, Path Length, and Memory results """
    metrics = ['Time', 'Path Length', 'Memory']
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    axes = axes.flatten()

    for i, metric in enumerate(metrics):
        sns.boxplot(x='Algorithm', y=metric, data=stats_df, ax=axes[i], palette='Set3')
        axes[i].set_title(f'{metric} Distribution')

    plt.tight_layout()
    plt.show()
```

```

def plot_pie_charts(stats_df):
    """ 3. Pie Charts: Shows the proportional consumption for each metric """
    metrics = ['Time', 'Path Length', 'Memory']
    fig, axes = plt.subplots(1, 3, figsize=(20, 7))

    avg_data = stats_df.groupby('Algorithm')[metrics].mean()
    colors = sns.color_palette('Set2')

    for i, metric in enumerate(metrics):
        axes[i].pie(avg_data[metric], labels=avg_data.index, autopct='%1.1f%%',
                    startangle=140, colors=colors)
        axes[i].set_title(f'Proportion of {metric}')

    plt.tight_layout()
    plt.show()

def plot_line_metrics(stats_df):
    """ 4. Line Plot: Analyzes performance stability (Time) across multiple runs """
    plt.figure(figsize=(12, 6))
    sns.lineplot(x=stats_df.index % 20, y='Time', hue='Algorithm', data=stats_df, marker='o')
    plt.title('Algorithm Performance Stability (Time per Run)')
    plt.xlabel('Run Number')
    plt.ylabel('Execution Time')
    plt.legend(title='Algorithm')
    plt.show()

```

4 Résultats et Analyse

Cette section présente les données quantitatives collectées lors de nos expérimentations. L'objectif est d'interpréter les performances de chaque algorithme en se basant sur les **moyennes** et les **écart-types** obtenus sur **20 simulations indépendantes**.

4.1 Présentation des Résultats Statistiques

Les tests ont été effectués sur un labyrinthe de taille **100 × 100** avec une densité d'obstacles de **30 %**. Le tableau ci-dessous résume les performances moyennes (*mean*) et la variabilité (*std*) pour le **temps d'exécution**, la **mémoire consommée** et l'**optimalité du chemin**.

STATISTICAL RESULTS (MEAN & STD)						
Algorithm	Time mean	std	Memory mean	std	Path Length mean	std
A*	0.0279	0.0259	0.2325	0.0980	200.0909	1.6404
BFS	0.0583	0.0201	0.6485	0.0337	199.5455	1.2933
DFS	0.0316	0.0033	0.5276	0.1479	1453.5714	150.6418
Greedy	0.0028	0.0006	0.0619	0.0001	245.2857	19.0238

Observations générales du tableau

- **Rapidité** : l'algorithme **Greedy** est de loin le plus rapide avec une moyenne de **0.0028 s**, suivi par **A*** (**0.0279 s**) et **DFS** (**0.0316 s**). **BFS** est le plus lent avec **0.0583 s**.
- **Efficacité mémoire** : **Greedy** consomme le moins de ressources avec **0.0619 MB**, tandis que **BFS** est le plus gourmand avec **0.6485 MB**, ce qui s'explique par son exploration exhaustive par couches.
- **Optimalité du chemin** : **BFS** et **A*** offrent les chemins les plus courts, avec des longueurs d'environ **199–200 unités**. En revanche, **DFS** produit des chemins extrêmement longs et sous-optimaux, avec une moyenne de **1453.57 unités**, confirmant sa nature de recherche en profondeur sans souci d'optimalité.

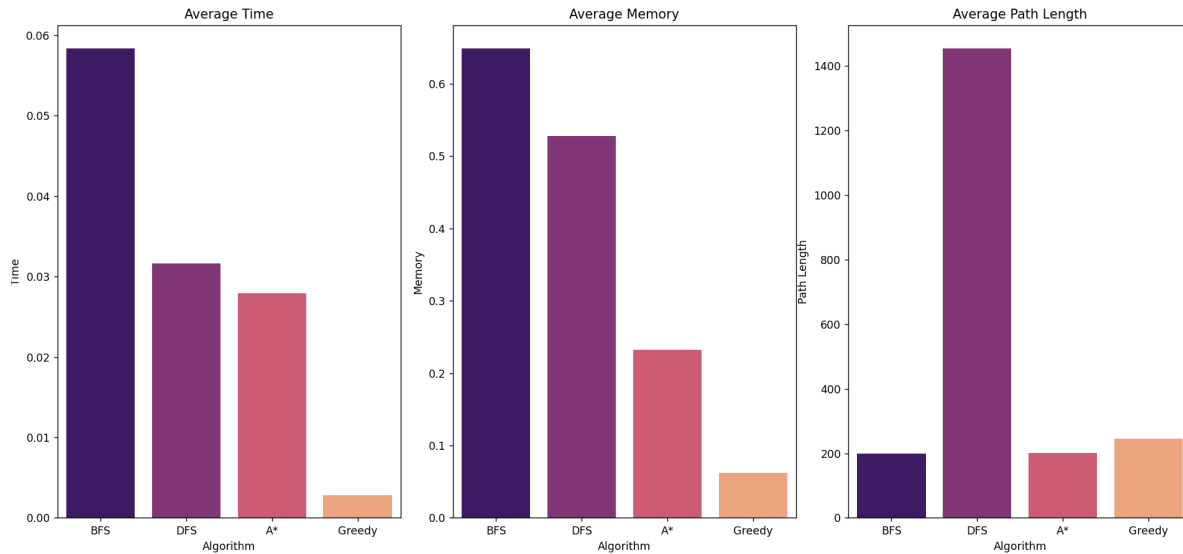
4.2 Analyse Comparative par Graphiques

Les graphiques suivants permettent d'interpréter visuellement les résultats statistiques. Ils montrent comment chaque algorithme se comporte face aux obstacles aléatoires du labyrinthe.

Note : les variations ou les pics observés dans les courbes s'expliquent par le fait que chaque labyrinthe est unique ; certains sont plus complexes ou plus denses que d'autres.

A. Analyse des Moyennes (Bar Plots)

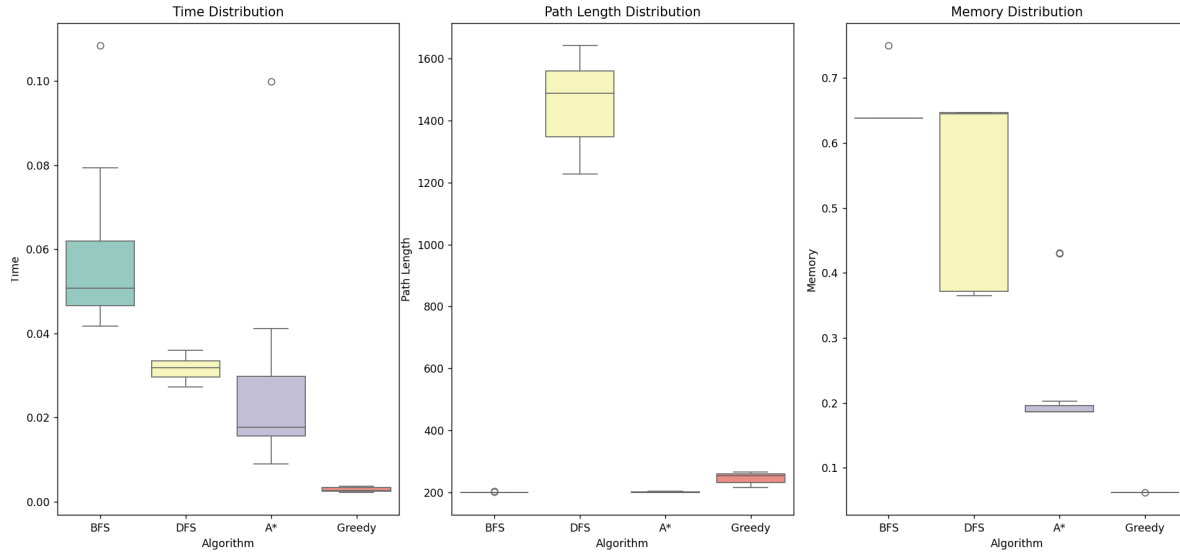
Le graphique des moyennes confirme la hiérarchie générale des performances observée dans le tableau statistique.



- **Temps et mémoire** : les algorithmes **Greedy** et **A*** dominent par leur efficacité. Greedy obtient les meilleurs temps d'exécution grâce à son choix exclusivement guidé par l'heuristique, tandis que A* conserve une performance élevée tout en intégrant le coût réel du chemin. À l'inverse, **BFS** affiche la consommation mémoire la plus élevée, car il doit conserver l'ensemble des nœuds de la frontière à chaque niveau d'exploration.
- **Longueur du chemin** : **BFS** et **A*** se situent au même niveau minimal, ce qui confirme leur capacité à produire des chemins optimaux ou quasi optimaux. **DFS**, en revanche, s'écarte fortement de cette tendance avec des chemins très longs, conséquence directe de son exploration en profondeur sans stratégie d'optimisation globale.
- **Comparaison globale** : cette visualisation montre clairement que la recherche informée, en particulier **A***, surpasse la recherche non-informée de type **BFS** dans presque toutes les métriques, notamment en temps et en mémoire, tout en conservant une qualité de chemin comparable.

B. Distribution et Variabilité (Box Plots)

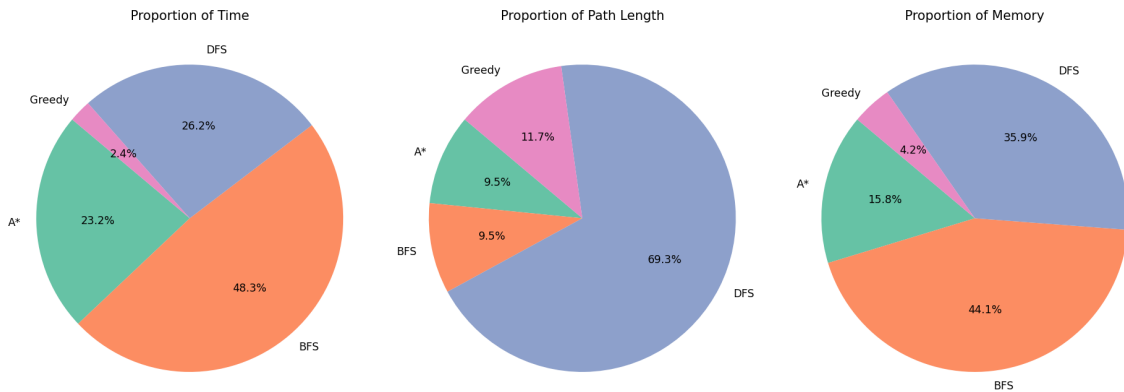
Les diagrammes à moustaches permettent d'étudier la dispersion des résultats sur les **20 exécutions** et d'évaluer la stabilité statistique des algorithmes.



- **Variabilité** : **A*** présente une dispersion plus importante dans le temps d'exécution que **Greedy**, car son score $f(n) = g(n) + h(n)$ dépend à la fois du chemin déjà parcouru et de l'heuristique. Ainsi, lorsque le labyrinthe généré contient des obstacles complexes, A* doit explorer davantage de possibilités avant d'atteindre la sortie.
- **Cas atypiques (outliers)** : la présence de points isolés pour **BFS** et **A*** indique que certaines configurations de labyrinthes sont particulièrement coûteuses. Ces cas correspondent généralement à des grilles où les obstacles imposent des détours importants ou réduisent fortement le nombre de passages disponibles.
- **Stabilité relative** : **Greedy** reste très compact dans sa distribution, ce qui traduit une grande rapidité et une faible variabilité. Cependant, cette stabilité temporelle ne garantit pas toujours l'optimalité du chemin, car Greedy privilégie uniquement la proximité heuristique avec l'objectif.

C. Efficacité d'Exploration (Pie Charts)

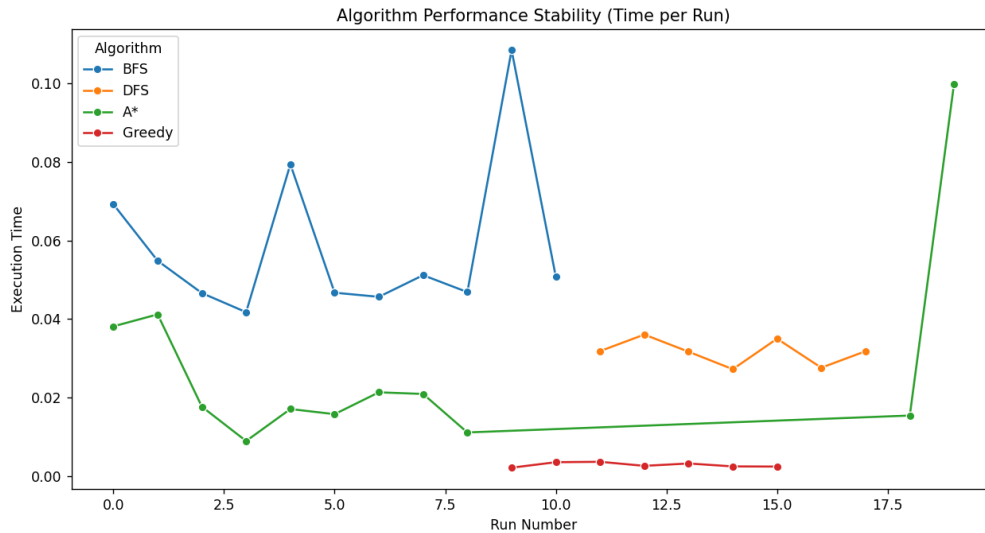
Les graphiques circulaires illustrent la répartition proportionnelle des ressources consommées et de l'espace de recherche exploité par chaque algorithme.



- **Temps et mémoire** : **BFS** occupe la plus grande part de la consommation de temps et de mémoire, avec plus de **44 %** dans les proportions observées. Ce résultat confirme son caractère exhaustif : l'algorithme explore une vaste partie de la grille avant de trouver la sortie, ce qui augmente fortement la taille de la frontière à stocker.
- **Longueur du chemin** : **DFS** domine la proportion de la longueur du chemin avec environ **69.3 %**. Cette part très élevée illustre visuellement son inefficacité à produire un trajet direct, car la stratégie en profondeur peut s'engager dans de longues branches avant de revenir vers la solution.
- **Apport de la recherche informée** : les parts plus réduites de **A*** montrent qu'une heuristique bien choisie, comme la distance de Manhattan, permet de limiter l'exploration inutile tout en conservant l'optimalité du chemin.

D. Stabilité au fil des Runs (Line Plot)

Le graphique linéaire montre l'évolution des performances au fil des **20 itérations**. Il permet d'observer la sensibilité des algorithmes aux variations introduites par la génération aléatoire des labyrinthes.



- **Pics et interruptions** : les ruptures ou pics visibles dans certaines courbes s'expliquent par la nature aléatoire de la génération du labyrinthe. La densité d'obstacles et la solvabilité effective de la grille peuvent varier d'un run à l'autre. Lorsqu'un labyrinthe est très complexe, ou lorsqu'aucun chemin valide n'est trouvé, les données peuvent devenir incomplètes ou afficher une hausse brutale du temps d'exécution.
- **Fiabilité** : **Greedy** reste l'algorithme le plus stable, avec une ligne presque plate au niveau le plus bas. Cette stabilité confirme son faible coût computationnel. **BFS**, au contraire, est le plus sensible aux variations de la structure du labyrinthe, car une augmentation de la complexité entraîne une expansion plus large de la frontière.
- **Lecture comparative** : **A*** présente un compromis plus robuste que **BFS**. Même lorsque la complexité augmente, il exploite l'information heuristique pour orienter la recherche, ce qui lui permet de dépasser la recherche aveugle dans presque toutes les métriques tout en conservant une solution de qualité.

5 Conclusion

Ce travail de recherche et d'implémentation nous a permis de tirer des conclusions claires sur l'efficacité des différentes stratégies de recherche :

- **Algorithmes non-informés** : l'algorithme **BFS** a démontré une fiabilité totale en garantissant l'**optimalité du chemin**, mais au prix d'une consommation mémoire importante due à une exploration exhaustive. À l'inverse, **DFS** s'est avéré inefficace pour la recherche de chemins courts, produisant des solutions très sous-optimales malgré sa faible empreinte mémoire.
- **Algorithmes informés** : la supériorité de la recherche informée est indéniable. L'algorithme **A*** s'impose comme le meilleur compromis : grâce à l'**heuristique de Manhattan**, il parvient à l'optimalité tout en explorant une fraction réduite de l'espace de recherche par rapport à **BFS**. De son côté, **Greedy Best-First Search** offre la rapidité d'exécution la plus élevée,

bien qu'il puisse parfois sacrifier l'optimalité du chemin.

En conclusion, ce travail souligne l'importance stratégique du choix de l'algorithme et de la fonction heuristique. Dans un contexte de résolution de problèmes complexes en **Intelligence Artificielle**, l'approche informée, illustrée par **A***, reste la méthode de référence pour concilier **précision, rapidité et gestion des ressources**.